Armors Labs

ASSET Token

Smart Contract Audit

- ASSET Token Audit Summary
- ASSET Token Audit
  - Document information
    - Audit results
    - Audited target file
  - Vulnerability analysis
    - Vulnerability distribution
    - Summary of audit results
    - Contract file
    - Analysis of audit results
      - Re-Entrancy
      - Arithmetic Over/Under Flows
      - Unexpected Blockchain Currency
      - Delegatecall
      - Default Visibilities
      - Entropy Illusion
      - External Contract Referencing
      - Unsolved TODO comments
      - Short Address/Parameter Attack
      - Unchecked CALL Return Values
      - Race Conditions / Front Running
      - Denial Of Service (DOS)
      - Block Timestamp Manipulation
      - Constructors with Care
      - Unintialised Storage Pointers
      - Floating Points and Numerical Precision
      - tx.origin Authentication
      - Permission restrictions

Armors Labs

ASSET Token Audit Summary

Project name : ASSET Token Contract

Project address: None

Code URL : https://github.com/RealFinOfficial/realtoken-erc20-contract/blob/main/contracts/RealToken.sol

Commit : 61cdb50b8a3ec85b7d87ecabcbdbe7328a13eb09

Project target : ASSET Token Contract Audit

Blockchain : Ethereum（ETH）

Test Status: Technically Verified

Audit Info

Audit NO : 0X202602070006

Audit Team : Armors Labs

Audit Proofreading: https://armors.io/#project-cases

ASSET Token Audit

The ASSET Token team asked us to review and audit their ASSET Token contract. We looked at the code and now publish our results.

Here is our assessment and recommendations, in order of importance.

Document information

Name	Some Auditors	Version	Date
ASSET Token Audit	Rock, Sophia, Rushairer, Rico, David, Alice	1.0.0	2026-02-07

Audit results

Note That:

1. The constructor mints the entire supply (1,000,000,000 * 10^18 tokens) to msg.sender (the deployer).
2. In `_requireValidAuthorization`, it uses `block.timestamp > validAfter` and `block.timestamp < validBefore`, without using `>=` and `<=`.

Disclaimer

Note that as of the date of publishing, the above review reflects the current understanding of known security patterns as they relate to the the audited contract. The above should not be construed as investment advice.

Armors Labs

Based on the widely recognized security status of the current underlying blockchain and smart contract, this audit report is valid for 3 months from the date of output.

Armors Labs Reports is not and should not be regarded as an "approval" or "disapproval" of any particular project or team. These reports are not and should not be regarded as indicators of the economy or value of any "product" or "asset" created by any team. Armors Labs do not cover testing or auditing the integration with external contract or services (such as Unicrypt, Uniswap, PancakeSwap etc'…)

Armors Labs Reports represent an extensive auditing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology. Armors Labs does not guarantee the safety or functionality of the technology agreed to be analyzed.

Armors Labs postulates that the information provided is not missing, tampered, deleted or hidden. If the information provided is missing, tampered, deleted, hidden or reflected in a way that is not consistent with the actual situation, Armors Labs shall not be responsible for the losses and adverse effects caused. Armors Labs Audits should not be used in any way to make decisions around investment or involvement with any particular project. These reports in no way provide investment advice, nor should be leveraged as investment advice of any sort.

Regulatory and Business Model Compliance: This audit is strictly a technical assessment of smart contract code integrity. Armors Labs does not evaluate, and explicitly disclaims any responsibility for, the legality, compliance, or regulatory standing of the underlying business model, including but not limited to multi-level marketing (MLM), referral structures, or financial licensing requirements in any jurisdiction.

Administrative Privilege Risk: The analyzed contract contains centralized administrative functions that allow designated addresses to modify critical parameters or access contract assets. Armors Labs is not responsible for any subsequent malicious actions, "rug pulls," or asset misappropriation performed by the contract owners or administrators using these privileged functions.

Exclusion of Business Logic: The scope of this audit is limited to the predefined technical vulnerability checklist. It does not include an assessment of the economic sustainability, tokenomics fairness, or the logical execution of incentive distribution mechanisms.

Infinite Risk Acknowledgment: Blockchain technology and smart contract security are evolving fields. This report does not guarantee that the contract is "unhackable" or free from future exploits, logical flaws, or zero-day vulnerabilities not covered by current industry-standard testing patterns.

Note on Audit Limitations: This assessment is a point-in-time, technical-only evaluation. It is restricted to identifying specific coding flaws and does not encompass financial, legal, or operational due diligence. The "COMPLETED/Technically Verified" status signifies the conclusion of the defined testing process, not a guarantee of total security or regulatory compliance.

Comprehensive Legal Disclaimer on Business & Regulation: Armors Labs' evaluation is purely technical. Test Status: COMPLETED/Technically Verified indicates only that the code was reviewed against a specific list of technical vulnerabilities. It DOES NOT verify the compliance of the project with anti-pyramid scheme laws, securities regulations, or anti-money laundering (AML) statutes. The existence of multi-tier referral logic or centralized admin overrides within the code is a technical observation and does not imply Armors Labs' endorsement of such features. Users and investors assume all risks associated with the project's operational and regulatory standing.

Customer Compliance Commitment: The Customer represents and warrants that the audited smart contract shall not be used for any illegal purposes or activities that violate the laws and regulations of the relevant jurisdictions. Armors Labs expressly disclaims any and all liability for any criminal or illicit activities conducted by the Customer or any third party utilizing this contract.

The audit does not constitute a formal verification or exhaustive security proof.

Armors Labs

In no event shall Armors Labs be liable for any direct, indirect, incidental, or consequential damages…

Armors Labs has no obligation to re-audit, monitor, or notify of future vulnerabilities discovered after the report date.

Automatic Invalidation: In the event that the audited contract is utilized for illegal purposes, including but not limited to fraud, money laundering, or unlicensed financial activities, this audit report shall be deemed null and void automatically, and Armors Labs shall bear no joint or several liability.

Audited target file

file	md5
./RealToken.sol	fdedffd108bfb14b47212388106a88d2

Vulnerability analysis

Vulnerability distribution

vulnerability level	number
Critical severity	0
High severity	0
Medium severity	0
Low severity	0

Summary of audit results

Vulnerability
Re-Entrancy
Arithmetic Over/Under Flows
Unexpected Blockchain Currency
Delegatecall
Default Visibilities
Entropy Illusion
External Contract Referencing
Short Address/Parameter Attack
Unchecked CALL Return Values
Race Conditions / Front Running
Denial Of Service (DOS)
Block Timestamp Manipulation
Constructors with Care

Armors Labs

Vulnerability
Unintialised Storage Pointers
Floating Points and Numerical Precision
tx.origin Authentication
Permission restrictions

Contract file

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./interface/IERC2612.sol";
import "./interface/IERC3009.sol";
import "@openzeppelin/contracts/utils/cryptography/MessageHashUtils.sol";
import "@openzeppelin/contracts/utils/cryptography/SignatureChecker.sol";
import "@openzeppelin/contracts/utils/cryptography/EIP712.sol";
import "@openzeppelin/contracts/utils/Nonces.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";

/**
 * @dev ERC-20 token with ERC-2612 permit and ERC-3009 transfer authorization support.
 *
 * Extends OpenZeppelin's {ERC20Burnable}, {EIP712}, and {Nonces} contracts,
 * and implements {IERC2612} and {IERC3009} interfaces. Supports both EOA (ECDSA)
 * and smart contract (ERC-1271) signature verification via {SignatureChecker}.
 */
contract RealToken is ERC20Burnable, EIP712, Nonces, IERC2612, IERC3009
{
    /**
     * @dev Sets the token name to "REAL", symbol to "ASSET", and EIP-712 domain name to "REAL" with
     * Mints 1,000,000,000 tokens (with 18 decimals) to the deployer.
     */
    constructor() ERC20("REAL", "ASSET") EIP712("REAL", "1") {
        _mint(msg.sender, 1_000_000_000 * 10**18);
    }

    /**
     * @dev The signature provided is not valid for the given signer and data.
     */
    error RealTokenInvalidSignature();

    /**
     * @dev Returns the number of decimals used to get the token's user representation.
     * @return The number of decimals (18).
     */
    function decimals() public pure override returns (uint8) {
        return 18;
    }

    /**
     * @dev Returns the EIP-712 domain separator for the current chain.
     * @return The domain separator as a bytes32 hash.
     */
    function DOMAIN_SEPARATOR() public view returns (bytes32) {
        return super._domainSeparatorV4();
    }
```

Armors Labs

```
    /**
     * @dev Validates a signature against the EIP-712 typed data hash.
     *
     * Uses {SignatureChecker} to support both EOA (ECDSA) and smart contract (ERC-1271) signatures.
     *
     * Requirements:
     *
     * - The signature must be valid for the given signer and encodeData.
     *
     * @param signer     The expected signer address
     * @param encodeData The EIP-712 struct hash to verify against
     * @param signature  The signature bytes
     */
    function _validateSignature(address signer, bytes32 encodeData, bytes memory signature) internal
        require(SignatureChecker.isValidSignatureNow(signer, MessageHashUtils.toTypedDataHash(DOMAIN_
    }

    // ============ ERC-2612 Implementation ============

    /// @dev keccak256("Permit(address owner,address spender,uint256 value,uint256 nonce,uint256 dead
    bytes32 public constant PERMIT_TYPEHASH = 0x6e71edae12b1b97f4d1f60370fef10105fa2faae0126114a169c6

    /**
     * @dev Internal implementation of the ERC-2612 permit.
     *
     * Validates the deadline, constructs the EIP-712 struct hash, verifies the signature,
     * and sets the allowance.
     *
     * Emits an {IERC20-Approval} event.
     *
     * Requirements:
     *
     * - block.timestamp must be less than or equal to deadline.
     * - The signature must be valid for the owner.
     *
     * @param owner     The token owner granting the allowance
     * @param spender   The address being approved to spend tokens
     * @param value     The amount of tokens approved
     * @param deadline  The time at which the signature expires (unix time)
     * @param signature The signature bytes
     */
    function _permit(address owner, address spender, uint256 value, uint256 deadline, bytes memory si
        if (block.timestamp > deadline) {
            revert ERC2612ExpiredSignature(deadline);
        }
        bytes32 structHash = keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, value, _useNonce(o
        _validateSignature(owner, structHash, signature);

        _approve(owner, spender, value);
    }

    /**
     * @dev See {IERC2612-permit}.
     */
    function permit(address owner, address spender, uint256 value, uint256 deadline, bytes memory sig
        _permit(owner, spender, value, deadline, signature);
    }

    /**
     * @dev See {IERC2612-permit}.
     *
     * Accepts the v, r, and s signature components and packs them into bytes.
     */
    function permit(address owner, address spender, uint256 value, uint256 deadline, uint8 v, bytes32
        _permit(owner, spender, value, deadline, abi.encodePacked(r, s, v));
    }
```

Armors Labs

```
// ============ ERC-3009 Implementation ============

/// @dev keccak256("TransferWithAuthorization(address from,address to,uint256 value,uint256 valid
bytes32 public constant TRANSFER_WITH_AUTHORIZATION_TYPEHASH = 0x7c7c6cdb67a18743f49ec6fa9b35f50d
/// @dev keccak256("ReceiveWithAuthorization(address from,address to,uint256 value,uint256 validA
bytes32 public constant RECEIVE_WITH_AUTHORIZATION_TYPEHASH = 0xd099cc98ef71107a616c4f0f941f04c32
/// @dev keccak256("CancelAuthorization(address authorizer,bytes32 nonce)")
bytes32 public constant CANCEL_AUTHORIZATION_TYPEHASH = 0x158b0a9edf7a828aad02f63cd515c68ef2f50ba

/// @dev Mapping of authorizer address => nonce => authorization state (true if used or canceled)
mapping(address => mapping(bytes32 => bool)) private _authorizationStates;

/**
 * @dev See {IERC3009-authorizationState}.
 */
function authorizationState(address authorizer, bytes32 nonce) external view returns (bool) {
    return _authorizationStates[authorizer][nonce];
}

/**
 * @dev See {IERC3009-transferWithAuthorization}.
 *
 * Accepts the v, r, and s signature components and packs them into bytes.
 */
function transferWithAuthorization(address from, address to, uint256 value, uint256 validAfter, u
    _transferWithAuthorization(from, to, value, validAfter, validBefore, nonce, abi.encodePacked(
}

/**
 * @dev See {IERC3009-receiveWithAuthorization}.
 *
 * Accepts the v, r, and s signature components and packs them into bytes.
 */
function receiveWithAuthorization(address from, address to, uint256 value, uint256 validAfter, ui
    _receiveWithAuthorization(from, to, value, validAfter, validBefore, nonce, abi.encodePacked(r
}

/**
 * @dev See {IERC3009-cancelAuthorization}.
 *
 * Accepts the v, r, and s signature components and packs them into bytes.
 */
function cancelAuthorization(address authorizer, bytes32 nonce, uint8 v, bytes32 r, bytes32 s) ex
    _cancelAuthorization(authorizer, nonce, abi.encodePacked(r, s, v));
}

/**
 * @dev See {IERC3009-transferWithAuthorization}.
 */
function transferWithAuthorization(address from, address to, uint256 value, uint256 validAfter, u
   _transferWithAuthorization(from, to, value, validAfter, validBefore, nonce, signature);
}

/**
 * @dev See {IERC3009-receiveWithAuthorization}.
 */
function receiveWithAuthorization(address from, address to, uint256 value, uint256 validAfter, ui
    _receiveWithAuthorization(from, to, value, validAfter, validBefore, nonce, signature);
}

/**
 * @dev See {IERC3009-cancelAuthorization}.
 */
function cancelAuthorization(address authorizer, bytes32 nonce, bytes memory signature) external
     _cancelAuthorization(authorizer, nonce, signature);
```

Armors Labs

```
    }

    /**
     * @dev Executes a transfer using a signed authorization.
     *
     * Validates the authorization timing and nonce, verifies the EIP-712 signature,
     * marks the authorization as used, and transfers tokens.
     *
     * Emits a {Transfer} event via {ERC20-_transfer}.
     * Emits an {AuthorizationUsed} event.
     *
     * @param from        Payer's address (Authorizer)
     * @param to          Payee's address
     * @param value       Amount to be transferred
     * @param validAfter  The time after which this is valid (unix time)
     * @param validBefore The time before which this is valid (unix time)
     * @param nonce       Unique nonce
     * @param signature   The signature bytes
     */
    function _transferWithAuthorization(address from, address to, uint256 value, uint256 validAfter,
        _requireValidAuthorization(from, nonce, validAfter, validBefore);
        _validateSignature(from, keccak256(abi.encode(TRANSFER_WITH_AUTHORIZATION_TYPEHASH, from, to,
        _useAuthorization(from, nonce);

        _transfer(from, to, value);
    }

    /**
     * @dev Executes a transfer using a signed authorization, with the additional requirement
     * that the caller must be the payee. This prevents front-running attacks where a third
     * party could observe the authorization and submit the transaction first.
     *
     * Emits a {Transfer} event via {ERC20-_transfer}.
     * Emits an {AuthorizationUsed} event.
     *
     * Requirements:
     *
     * - to must be equal to msg.sender.
     *
     * @param from        Payer's address (Authorizer)
     * @param to          Payee's address (must be msg.sender)
     * @param value       Amount to be transferred
     * @param validAfter  The time after which this is valid (unix time)
     * @param validBefore The time before which this is valid (unix time)
     * @param nonce       Unique nonce
     * @param signature   The signature bytes
     */
    function _receiveWithAuthorization(address from, address to, uint256 value, uint256 validAfter, u
        require(to == msg.sender, ERC3009CallerMustBeThePayee());
        _requireValidAuthorization(from, nonce, validAfter, validBefore);
        _validateSignature(from, keccak256(abi.encode(RECEIVE_WITH_AUTHORIZATION_TYPEHASH, from, to,
        _useAuthorization(from, nonce);

        _transfer(from, to, value);
    }

    /**
     * @dev Cancels an authorization, preventing it from being used in the future.
     *
     * Emits an {AuthorizationCanceled} event.
     *
     * Requirements:
     *
     * - The authorization must not have been already used or canceled.
     *
     * @param authorizer Authorizer's address
```

Armors Labs

```
     * @param nonce      Nonce of the authorization to cancel
     * @param signature  The signature bytes
     */
    function _cancelAuthorization(address authorizer, bytes32 nonce, bytes memory signature) internal
        require(!_authorizationStates[authorizer][nonce], ERC3009UsedOrCanceledAuthorization());
        _validateSignature(authorizer, keccak256(abi.encode(CANCEL_AUTHORIZATION_TYPEHASH, authorizer
        _authorizationStates[authorizer][nonce] = true;
        emit AuthorizationCanceled(authorizer, nonce);
    }

    /**
     * @dev Validates the timing and nonce of an authorization.
     *
     * Requirements:
     *
     * - block.timestamp must be greater than validAfter.
     * - block.timestamp must be less than validBefore.
     * - The nonce must not have been previously used or canceled.
     *
     * @param authorizer Authorizer's address
     * @param nonce      Nonce of the authorization
     * @param validAfter The time after which this is valid (unix time)
     * @param validBefore The time before which this is valid (unix time)
     */
    function _requireValidAuthorization(address authorizer, bytes32 nonce, uint256 validAfter, uint25
        require(block.timestamp > validAfter, ERC3009NotYetValidAuthorization(validAfter));
        require(block.timestamp < validBefore, ERC3009ExpiredAuthorization(validBefore));
        require(!_authorizationStates[authorizer][nonce], ERC3009UsedOrCanceledAuthorization());
    }

    /**
     * @dev Marks an authorization nonce as used.
     *
     * Emits an {AuthorizationUsed} event.
     *
     * @param authorizer Authorizer's address
     * @param nonce      Nonce to mark as used
     */
    function _useAuthorization(address authorizer, bytes32 nonce) private {
        _authorizationStates[authorizer][nonce] = true;
        emit AuthorizationUsed(authorizer, nonce);
    }

}
```

Armors Labs

Analysis of audit results

Re-Entrancy

- **Description:**
  One of the features of smart contracts is the ability to call and utilise code of other external contracts. Contracts also typically handle Blockchain Currency, and as such often send Blockchain Currency to various external user addresses. The operation of calling external contracts, or sending Blockchain Currency to an address, requires the contract to submit an external call. These external calls can be hijacked by attackers whereby they force the contract to execute further code (i.e. through a fallback function) , including calls back into itself. Thus the code execution "re-enters" the contract. Attacks of this kind were used in the infamous DAO hack.

Arithmetic Over/Under Flows

- **Description:**
  The Virtual Machine (EVM) specifies fixed-size data types for integers. This means that an integer variable, only has a certain range of numbers it can represent. A uint8 for example, can only store numbers in the range [0,255]. Trying to store 256 into a uint8 will result in 0. If care is not taken, variables in Solidity can be exploited if user input is unchecked and calculations are performed which result in numbers that lie outside the range of the data type that stores them.

Unexpected Blockchain Currency

- **Description:**
  Typically when Blockchain Currency is sent to a contract, it must execute either the fallback function, or another function described in the contract. There are two exceptions to this, where Blockchain Currency can exist in a contract without having executed any code. Contracts which rely on code execution for every Blockchain Currency sent to the contract can be vulnerable to attacks where Blockchain Currency is forcibly sent to a contract.

Delegatecall

- **Description:**
  The CALL and DELEGATECALL opcodes are useful in allowing developers to modularise their code. Standard external message calls to contracts are handled by the CALL opcode whereby code is run in the context of the external contract/function. The DELEGATECALL opcode is identical to the standard message call, except that the code executed at the targeted address is run in the context of the calling contract along with the fact that msg.sender and msg.value remain unchanged. This feature enables the implementation of libraries whereby developers can create reusable code for future contracts.

Default Visibilities

- **Description:**
  Functions in Solidity have visibility specifiers which dictate how functions are allowed to be called. The visibility determines whBlockchain Currency a function can be called externally by users, by other derived contracts, only internally or only externally. There are four visibility specifiers, which are described in detail in the Solidity Docs. Functions default to public allowing users to call them externally. Incorrect use of visibility specifiers can lead to some devestating vulernabilities in smart contracts as will be discussed in this section.

Entropy Illusion

- **Description:**
  All transactions on the blockchain are deterministic state transition operations. Meaning that every transaction modifies the global state of the ecosystem and it does so in a calculable way with no uncertainty. This ultimately means that inside the blockchain ecosystem there is no source of entropy or randomness. There is no rand() function in Solidity. Achieving decentralised entropy (randomness) is a well established problem and many ideas have been proposed to address this (see for example, RandDAO or using a chain of Hashes as described by Vitalik in this post).

External Contract Referencing

Armors Labs

- **Description:**
  One of the benefits of the global computer is the ability to re-use code and interact with contracts already deployed on the network. As a result, a large number of contracts reference external contracts and in general operation use external message calls to interact with these contracts. These external message calls can mask malicious actors intentions in some non-obvious ways, which we will discuss.

Unsolved TODO comments

- **Description:**
  Check for Unsolved TODO comments

Short Address/Parameter Attack

- **Description:**
  This attack is not specifically performed on Solidity contracts themselves but on third party applications that may interact with them. I add this attack for completeness and to be aware of how parameters can be manipulated in contracts.

Unchecked CALL Return Values

- **Description:**
  There a number of ways of performing external calls in solidity. Sending Blockchain Currency to external accounts is commonly performed via the transfer() method. However, the send() function can also be used and, for more versatile external calls, the CALL opcode can be directly employed in solidity. The call() and send() functions return a boolean indicating if the call succeeded or failed. Thus these functions have a simple caveat, in that the transaction that executes these functions will not revert if the external call (intialised by call() or send()) fails, rather the call() or send() will simply return false. A common pitfall arises when the return value is not checked, rather the developer expects a revert to occur.

Race Conditions / Front Running

- **Description:**
  The combination of external calls to other contracts and the multi-user nature of the underlying blockchain gives rise to a variety of potential Solidity pitfalls whereby users race code execution to obtain unexpected states. Re-Entrancy is one example of such a race condition. In this section we will talk more generally about different kinds of race conditions that can occur on the blockchain. There is a variety of good posts on this subject, a few are: Wiki - Safety, DASP - Front-Running and the Consensus - Smart Contract Best Practices.

Denial Of Service (DOS)

- **Description:**
  This category is very broad, but fundamentally consists of attacks where users can leave the contract inoperable for a small period of time, or in some cases, permanently. This can trap Blockchain Currency in these contracts forever, as was the case with the Second Parity MultiSig hack

Block Timestamp Manipulation

- **Description:**
  Block timestamps have historically been used for a variety of applications, such as entropy for random numbers

Armors Labs

(see the Entropy Illusion section for further details), locking funds for periods of time and various state-changing conditional statements that are time-dependent. Miner's have the ability to adjust timestamps slightly which can prove to be quite dangerous if block timestamps are used incorrectly in smart contracts.

Constructors with Care

- **Description:**
  Constructors are special functions which often perform critical, privileged tasks when initialising contracts. Before solidity v0.4.22 constructors were defined as functions that had the same name as the contract that contained them. Thus, when a contract name gets changed in development, if the constructor name isn't changed, it becomes a normal, callable function. As you can imagine, this can (and has) lead to some interesting contract hacks.

Unintialised Storage Pointers

- **Description:**
  The EVM stores data either as storage or as memory. Understanding exactly how this is done and the default types for local variables of functions is highly recommended when developing contracts. This is because it is possible to produce vulnerable contracts by inappropriately intialising variables.

Floating Points and Numerical Precision

- **Description:**
  As of this writing (Solidity v0.4.24), fixed point or floating point numbers are not supported. This means that floating point representations must be made with the integer types in Solidity. This can lead to errors/vulnerabilities if not implemented correctly.

tx.origin Authentication

- **Description:**
  Solidity has a global variable, tx.origin which traverses the entire call stack and returns the address of the account that originally sent the call (or transaction). Using this variable for authentication in smart contracts leaves the contract vulnerable to a phishing-like attack.

Permission restrictions

- **Description:**
  Contract managers who can control liquidity or pledge pools, etc., or impose unreasonable restrictions on other users.

Armors Labs

armors.io

contact@armors.io